Encode & Decode Strings

Non-ASCII delimiter [T.C. - O(N), S.C. - O(1) → answer space]

→ In the problem statement, it is mentioned that the input string only consists of the 256 ASCII standard characters.
→ We can use this information to our advantage, and choose a unicode character, outside the ASCII boundary, for example, an emoji '🔥' as a delimiter while joining & splitting strings! Under this scenario, we wouldn't have to worry if the chosen delimiter could be a part of any original input string, leading to erroneous splitting

Catch! → Still, if not been careful, empty input strings could get lost inside the encoded string!

→ And using .split(), there would be no way to know, whether there were any empty string also present!

→ So, to not loose this information, we add a different unicode character, whenever an empty string ("") is encountered!

Limitations → We were lucky that the input strings contained only ASCII characters.
→ If the entire UNICODE charset was allowed, we would have to apply a stricter delimiter!

Escaping (very interesting...)

→ We definitely want to use a stricter de-limiter such that any input can be taken care of.

→ In computer programming, using the escape character '\' is used very commonly

→ We can take "/:" as the choice for our delimiter.

→ Naturally the question arises, what if the input string contains our delimiter string?

For example → str = ["hello", "wor/:ld", "nice"]
encoded string → "hello/:wor/:ld/:nice"

unfortunately decoded strings → ["hello", "wor", "ld", "nice"]

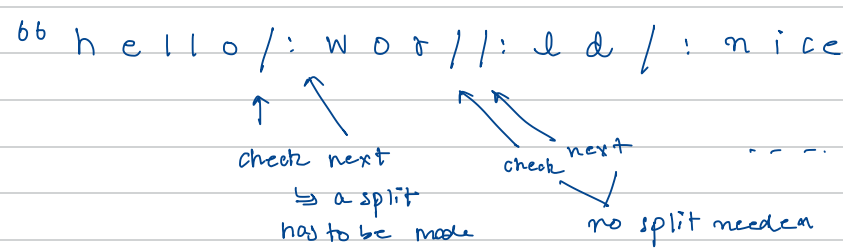
our original string got effected!

→ So, are we back to square one? Fortunately, no and we can take care of it by adding an escape sequence '\/', whenever there is '\' present in the original string!

↳ ∴ Now, str' = ["hello" "wor//:ld", "nice"]
encoded string → hello/:wor//:ld/:nice

Decoding → due to the nature of our de-limiter, we can't directly use the split() method.

→ We have to tweak our iterat^n while decoding!



→ Basically, whenever we encounter '/' & ':' in a sequence, it signifies the characters till yet are part of a different word

→ Otherwise, we assume that '/' is a part of the natural string & continue to append

Chunking (delightfully interesting)

↳ We would use the informat^n of each string to convert into something unique.

↳ We will use each string's length and any delimiter combined with the original string

E.g → "hello/:" → "7#hello/:" → modified string

length, delimiter

↳ these individually converted strings are called chunks & the process is called chunking

↳ In real world, data packets are created using a similar chunking strategy!!

Decoding

↳ We just have to extract the length of a chunk, extract the string, & keep moving to the next chunk!

↳ Yes, is that's simple

Pseudo-code

```
String encoded;

for(int i → encoded) {
    int j = i
    while(c[j] != '#') {
        j++;
    }
    length = encoded.substring(i, j) → Integer.parseInt()
    word = encoded.substring(j+1, j+1+length);
    answer.add(word);
}
```

Thanks for reading!
Keshav Dutt Kaushik